flownexus Documentation

Release v0.1.1-63-gf68b8ef

Jonas Remmert, Akarshan Kapoor

Mar 15, 2026

DOCUMENTATION

Index **37**

Introduction

Welcome to flownexus, your comprehensive solution for small to medium scale IoT deployments. This platform is designed to streamline the setup, configuration, and use of IoT devices, making it easy to manage your IoT ecosystem.

This project aims to provide a minimal and standalone Open Source IoT platform. The focus is on systems with a deployment of up to a few thousand devices; it does not aim to be scalable to millions of devices. The goal is to build a system that receives, for example, regular temperature samples from remote IoT devices. These temperature samples will be sent to a backend and stored in a database. The data is then visualized in a web application.

Our vision for flownexus is to create a user-friendly, efficient, and robust Open Source platform for IoT management. We aim to simplify the complexities of IoT deployments, making them accessible and manageable for businesses of all sizes.

CHAPTER

ONE

OVERVIEW

This documentation describes a local server-based IoT system that leverages the Lightweight Machine to Machine (LwM2M) protocol to communicate between IoT devices running Zephyr OS and a backend server using Django. The system does not depend on external cloud services and is designed to operate fully within a local environment.

1.1 Features

- No dependencies of external services like AWS, MQTT brokers or similar. The system has to be able to run in a local environment.
- The main focus is the LwM2M protocol and the communication between Zephyr and the server.
- The system should show how to add data to a database and visualize it in a web application.
- The web application has to support a secure login and basic user management.
- Certificate based authentication and encryption.
- Server to read and observe resources from IoT Device.
- Server to write resources to IoT Devices.
- OTA Update.
- Receive logs from IoT Devices.

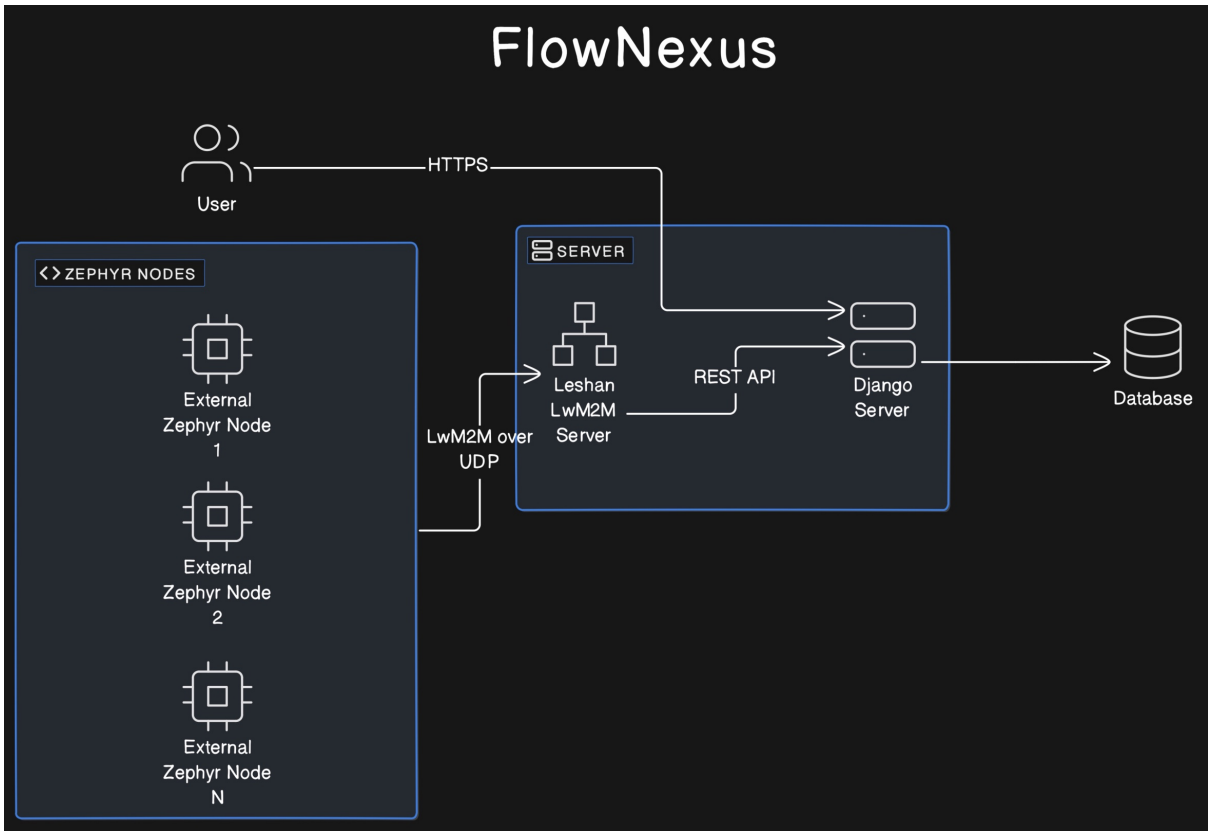
1.2 Key Components

1.2.1 IoT Devices

These are typically resource-constrained devices (limited energy and storage capabilities) that run Zephyr OS. They communicate with the server using the UDP protocol to ensure constant connectivity, even in low power states. These devices are programmed to handle tasks like temperature sensing, location tracking, or any other telemetry based on Zephyr applications. They communicate via the UDP protocol to maintain a continuous link without needing frequent reconnections. The IoT device has to support LwM2M. Mainly systems that run Zephyr should be compatible. The application can e.g. be used with a dedicated nRF9160 device or via a simulation like native_sim or in Renode. The nRF9160 is a low power LTE-M and NB-IoT SoC that runs Zephyr OS. UDP is used as transport protocol as it allows to keep devices connected even when the device is sleeping for extended periods (TCP would require a new connection setup typically after a few minutes)

1.2.2 LwM2M Server

The LwM2M server plays a critical role in managing communications with IoT devices. It handles data transmission, device monitoring, and control commands, ensuring that devices can report their telemetry data, receive configuration updates, and execute commands sent by the server. The server's efficient management of these tasks is essential for maintaining reliable communication with numerous IoT devices, especially those with limited power and processing resources.


FlowNexus
User
HTTPS
ZEPHYR NODES
External Zephyr Node 1
External Zephyr Node 2
External Zephyr Node N
LwM2M over UDP
SERVER
Leshan LwM2M Server
REST API
Django Server
Database

CHAPTER

TWO

ARCHITECTURE

This section describes the architecture of individual components and how they interact with each other.

2.1 Django

The Django server hosts the REST API, manages the database, and provides a web interface for data visualization and user management. This server acts as the backend infrastructure that supports the entire IoT ecosystem by:

- **REST API**: Facilitates communication between the IoT devices and the server. The API endpoints handle requests for data uploads from devices, send control commands, and provide access to stored telemetry data.
- **Database Management**: Stores all the telemetry data, device information, user accounts, and configuration settings. The database ensures that data is organized, searchable, and retrievable in an efficient manner.
- **Web Interface**: Offers a user-friendly interface for data visualization and management. Through this interface, users can monitor device status, visualize telemetry data in real-time, manage user accounts, and configure device settings.

2.1.1 Build and Run

The Django server can also run locally, without the need of a container. Make sure to create a virtual environment and install the requirements:

```
host:~$ source venv/bin/activate
host:~$ cd workspace/flownexus/server/django
host:~/workspace/flownexus/server/django$ pip install -r requirements.txt
host:~/workspace/flownexus/server/django$ ./django_start.sh
```

The Django server should now be up and running under the following URL: `http://localhost:8000/admin`. The admin login is `admin` and the password

Testing

For testing documentation, see *Devtools & Testing*.

2.1.2 Database Model

The database model is the core of the Django server. It aims to store information according to the LwM2M resource model. The advantage is that data can be stored in a generic way and the server can be extended with new resources without changing the database schema.

The server application logic only has to handle higher level events. Those events are situations where multiple resources are associated (e.g. Temperature, Pressure, Humidity, Acceleration). Those multiple resources are linked in the event, together with a timestamp. The event itself is represented by the database model in a generic way, several custom event types can be created by the application logic.

An Entity Relationship Diagram (ERD) is a visual representation of the database schema. It is automatically generated from the Django models. `sensordata` is the Django app that contains the application logic.

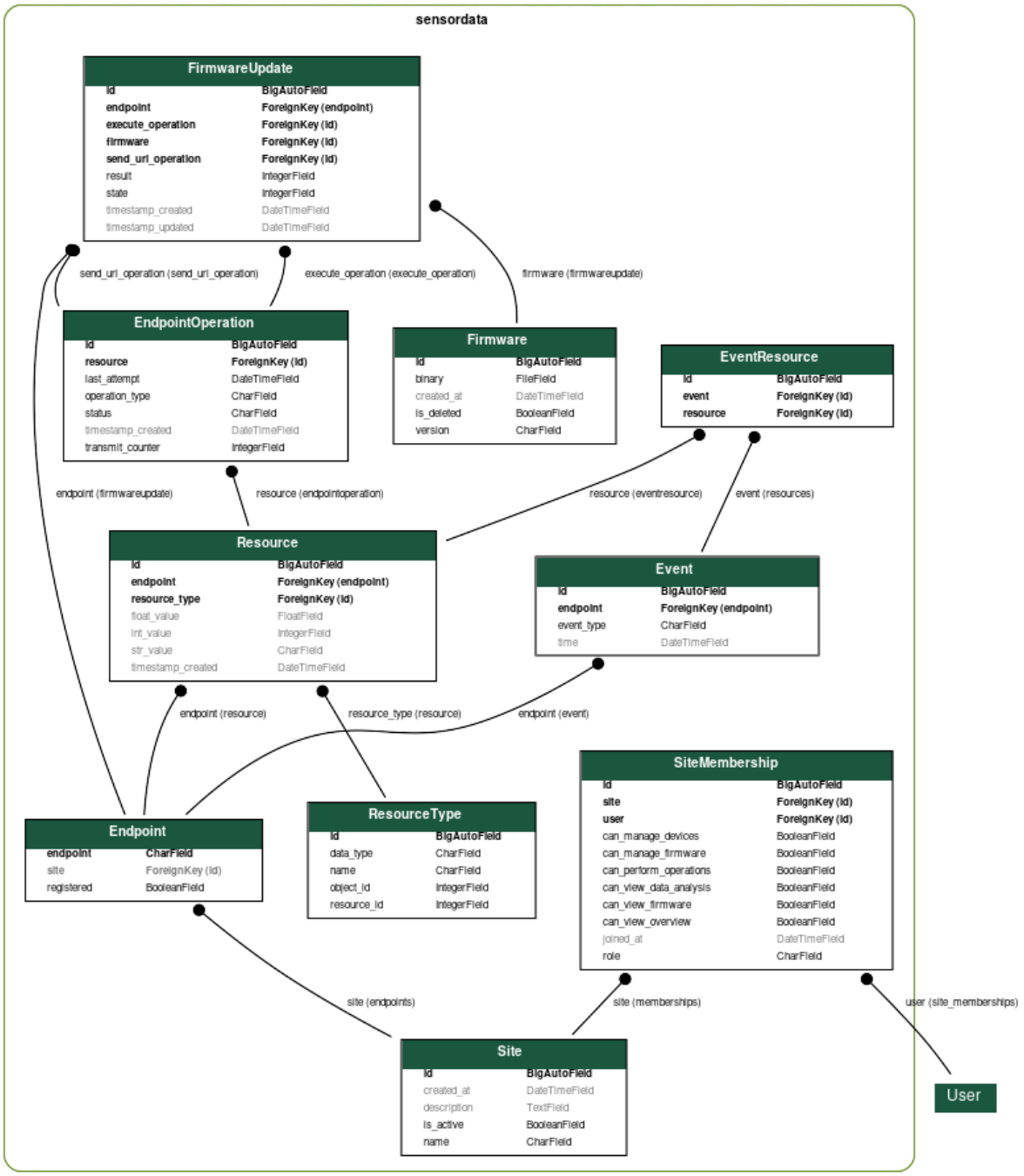


Fig. 1: Entity Relationship Diagram generated from Django models

Endpoint
: IoT devices using the LwM2M protocol in the network, identifiable by a unique name, e.g. `urn:imei:123456789012345`.

ResourceType
: Defines resource data points comprehensively, annotating each with a unique object-resource ID combination, a descriptive name, and specifying the expected data type.

Resource

A specific piece of data or functionality within an LwM2M Object. Resources represent attributes or actions and are identified by Resource IDs within an Object.

In the database Model a Resource represents an individual data value from one IoT device, annotated with timestamps, applicable data types, and linked to both the device and resource type for which the data is relevant.

Event
A collection for significant occurrences reported by endpoints. Events and Resources are linked to each other via EventResource table.

EventResource
Acts as a junction table forming a many-to-many relationship between events and their constituent resources, enabling flexible association without direct modification to the core events or resources tables.

EndpointOperation
Represents actionable commands or processes targeted at endpoints, tracking the operation type, status, and scheduling through timestamps, also detailing the transmission attempts and last action.

Firmware
Stores metadata about firmware binaries that are available for devices to download and install. Each record includes a version identifier, the name of the file, a URL from where the device can retrieve the firmware, and timestamps for tracking when each firmware record was created and last updated.

irmwareUpdate
keeps track of the execution of firmware updates for each endpoint. It adds references to the two required resources from server to endpoint (Send URI, execute Update). Furthermore it adds a field for the State and the Result of an update.

ite
Represents an organizational unit for grouping devices within a single deployment. Devices (Endpoints) belong to exactly one Site, and users can be assigned to one or multiple Sites with specific roles and permissions.

iteMembership
Links users to Sites with role-based access control. Defines permissions for viewing data, managing firmware, performing device operations, and administrative functions within a Site context.

2.1.3 Access Control

The system supports multi-site deployments with role-based access control. See *Access Control* for detailed documentation on:

- Site organization for grouping devices
- Role hierarchy (Site User, Site Admin, Global Admin)
- Permission system and feature access
- User management workflows
- Mock environment scenarios for testing

2.2 Leshan

2.2.1 Build and Run

The Leshan server can also run locally, without the need of a container.

```
host:~$ sudo apt update
host:~$ sudo apt install openjdk-17-jdk maven
host:~$ source venv/bin/activate
host:~$ cd workspace/flownexus/server/leshan
host:~/workspace/flownexus/server/leshan$ ./leshan_build_run.sh
```

The Leshan server should now be up and running under the following URL: `http://localhost:8080`.

2.2.2 Leshans Role in flownexus

Lehsan is considered a production ready LwM2M server and is used in many commercial products. It is used as an LwM2M server in flownexus. It is responsible for registering and managing *endpoint*. There is no application specific logic implemented in Leshan related Java code within flownexus. The goal is to have all application logic within the Django application.

Leshan can be seen as a proxy between the LwM2M endpoints Django. There is no direct communication between Django and endpoints.

2.2.3 LwM2M Observe

LwM2M observe
: The *LwM2M observe* operation allows an LwM2M Server to monitor changes to specific Resources, Resources within an Object Instance, or all Object Instances of an Object on an LwM2M Client. An endpoint must remember these observation requests until re-registration.

Single and CompositeObserve
: Leshan supports two types of observe requests: Single and CompositeObserve. SingleObserve is a simple observe request for a single resource. CompositeObserve is a more complex observe request that subscribes to multiple *Resources* of one *Object*.

Once an endpoint registers to Leshan, Leshan will initiate an observe request to selected resources. With every new registration of an endpoint, the observe requests are re-initiated.

Note

The observe initiation will move from Leshan to Django with future versions. Django is informed about new registrations and will then initiate the observe requests. This allows a more centralized approach and better control over the observe requests via the database model.

For testing, the Leshan server currently initiates the following two observe requests:

- SingleObserve (Temperature Sensor Value): `{3303, 0, 5700}`
- CompositeObject (Custom Object Id): `{10300}`

2.3 Communication, Interfaces

The communication between IoT devices and Leshan is specified by the OMA LwM2M standard:

- LwM2M core specification v1.1.1
- LwM2M transport binding v1.1.1

The standard describes how the LwM2M server (Leshan) works, however, it does not describe how to connect a backend server to Leshan. The backend is responsible for storing the data in a database and implementing application logic. A frontend can access the data in the database and visualize outward facing user interfaces. Leshan acts as a gateway between Endpoints and the backend. There should be no application specific logic implemented in Leshan.

In order to communicate and exchange data, both components (Leshan LwM2M Server and Django) post data to each other's ReST APIs. Communication is typically triggered by IoT devices sending data or the user/application requesting data from devices.

2.3.1 Data Flow: Endpoint -> Backend

All communication from Endpoints to the server flows through Leshan. Leshan interprets this data according to the LwM2M protocol and generates a ReST API call to the backend server. The backend server then stores the data in the database.

Django hosts the ReST API that Leshan posts to. Serializers, a part of Django, deserialize the incoming data and store it in the database according to the database model.

Leshan Data Format

There are two types of data that Leshan sends to the backend, single resource and composite resource format. The two ReST API endpoints that Leshan posts to are available under the following URLs:

- `/leshan_api/resource/single`: single resource format.
- `/leshan_api/resource/composite`: composite resource format.

For more details, please theck the `Internal Django` API documentation.

Listing 1: Single Resource Format (3303/0/5700)

```
{
   "ep": "qemu_x86",
   "obj_id": 3303,
   "val": {
       "kind": "singleResource",
       "id": 5700,
       "type": "FLOAT",
       "value": "24.899181214836236"
   }
}
```

Listing 2: Composite Resource Format (3/0/0..17)

```
{
    "ep" : "qemu_x86",
    "val" : {
        "instances" : [ {
            "kind" : "instance",
            "resources" : [ {
                "kind" : "singleResource",
                "id" : 0,
                "type" : "STRING",
                "value" : "Zephyr"
            }, {
                "kind" : "multiResource",
                "values" : {
                    "0" : "1",
                    "1" : "5"
                },
            },
                "kind" : "singleResource",
                "id" : 17,
                "type" : "STRING",
                "value" : "qemu_x86"
```

(continues on next page)

(continued from previous page)

```
            } ],
            "id" : 0
        } ],
        "kind" : "obj",
        "id" : 3
    }
}
```

The marked lines in the composite resource format show where Object ID, Instance ID and Resource ID are located. A composite resource format can consist of multiple Object IDs.

Warning

Currently multiResources are not supported and will be ignored. MultiResource dataypes are e.g. used for Voltage range (min, max).

Registration Updates

Leshan maintains registrations of endpoints. An endpoint can be registered or unregistered. To maintain its registration, an endpoint must send an update to Leshan regularly. If an endpoint does not send an update within the specified duration, it is considered offline and will be unregistered by Leshan.

Those registration events are encapsulated into an LwM2M Object and sent to the backend server. The backend server stores the registration events in the database. This allows to use the generic database model for the registration events as well. All received registration events are stored in the database and can be used for statistics.

All registration events are maintained in the custom LwM2M Object ID `10240`:

- `10240/0/0`: Endpoint **registered** to Leshan.
- `10240/0/1`: Endpoint **unregistered** from Leshan.
- `10240/0/2`: Endpoint **updated** its registration.

2.3.2 Data Flow: Backend -> Endpoint

Endpoints often operate in queue mode, meaning they are not always online. The LwM2M Server is aware of the current status of a device (Online/Offline) and communicates this status to the backend server. Leshan does not queue pending data that should be sent to the device when it comes online. The backend server must handle this by itself so it has to have a representation of the current status of each device as well as the data to be send. The resource table `EndpointOperation` is used to store pending operations that should be sent to the endpoint while it is online.

Once an endpoint updates it's registration (LwM2M Update Operation) Leshan notifies the backend. The backend checks the `EndpointOperation` table for pending operations and sends them to the device by posting to the Leshan hosted ReST API. Leshan keeps the post call open until the device acknowledges the operation or a timeout is generated. Endpoints can be slow to respond (several Seconds), so the backend has to handle the ReST API call in an asynchronous manner. By only sending data to endpoints while they are online, the backend can be sure that the ReST API calls are not open for a long time.

Asynchronous Communication

Given that endpoints are comparably slow to respond, handling communication asynchronously is essential for efficient operation. This can be effectively managed using Celery, a distributed task queue. When Leshan notifies the backend of an endpoint status update, Celery can be used to handle the long-running API calls, ensuring that the backend remains responsive and scalable. As the backend communicates with many endpoints simultaneously, an efficient queing mechanism is essential to ensure that the system remains responsive and scalable.

Before the backend executes the API call, it updates the endpointOperation status to `SENDING`, indicating an ongoing operation.

Once the API call is complete the database will be updated with the result (e.g. `CONFIRMED`, `FAILED`, `QUEUED`) depending on the result of the request. The `FAILED` status is assigned after 3 attempts. Retransmissions are triggered when the endpoint updates it's registration the next time.

Example Communication

The following example shows how the backend server can send a firmware download link resource `Package URI 5/0/1` to an endpoint:

1. User creates new `EndpointOperation`: resource path `5/0/1`, value `https://url.com/fw.bin`.
2. Backend checks endpoint online status.
3. If endpoint is offline, no further action is taken right away.
4. Endoint comes online, Leshan sends update to the backend.
5. Backend checks `EndpointOperation` table for pending operations for the endpoint.
6. Finds pending operation, send resource to endpoint via the Leshan ReST API.
7. Pending operation is marked `completed` if the endpoint acknowledges the operation.

2.4 Over the Air Updates

Over the Air (OTA) updates are a way to update the firmware of a device remotely. This is a mandatory feature for any IoT device. The device must be able to update its firmware without any physical user intervention.

flownexus implements OTA updates according to the LwM2M protocol. The LwM2M protocol specifies how to update the firmware of a device. The OTA process is implemented by sending a link to a firmware via LwM2M object 5/0/1 - Package URI. This method is called Pull via HTTP(s) and is faster, compared to a transfer via CoAP. Updates via CoAP blockwise transfer to download the firmware are not supported.

A https server is usually be hosted under a subdomain. The default is `https://fw.flownexus.org`. This makes it easy to use a self-signed certificate for the https server. For details, check chapter *Setup a Virtual Server*.

More information about a possible firmware implementation can be found in the `lwm2m_client` firmware sample at `flownexus/devtools/zephyr/client/src/firmware_update.c`.

Note

Firmware Updates are implemented in the LwM2M server and the example firmware. A set up a PKI with self-signed certificates is required for the server to communicate with the device via https. This setup will be described in upcoming releases shortly.

2.4.1 Update process of a Firmware

The update process is described in detail in the LwM2M core specification v1.1.1 Chapter *E.6 LwM2M Object: Firmware Update*. This chapter summarizes the update process:

1. flownexus sends a link to the firmware to the device via LwM2M object 5/0/1 - Package URI. It is a relative link to the firmware on the server, e.g. `/binaries/v0.1.0.bin`. The host `https://fw.flownexus.org` is fixed in firmware.
2. The device downloads the firmware from the server via http(s) and sets the state to `STATE_DOWNLOADING`.
3. The device sets the state to `STATE_DOWNLOADED` after the download is complete.

4. flownexus executes the command 5/0/2 - Update to the device.
5. The device sets the state to `STATE_UPDATING` upon receiving the command.
6. The device updates the firmware, usually by performing a reboot.
7. The device registers and sends the firwmare version to flownexus via resource `Firmware Version - 3/0/3`. Depending if the update was successful or not, flownexus sets the result and state of the update in the database accordingly.

This process ensures that errors during the update process are handled and communicated to the backend.

States of a OTA Update

There are 4 device states during an OTA update. flownexus tracks the state of the device during the update process in the database (FirmwareUpdate table). Once the update is completed (failed or successful), the state is set always set to `STATE_IDLE`.

Table 1: Device States during an OTA Update

State	Description
STATE_IDLE	device is idle
STATE_DOWNLOADING	device is downloading
STATE_DOWNLOADED	download is complete
STATE_UPDATING	device is updating

Result Codes for an OTA Update

For every OTA update, a result is generated after the update failed or successful. There can be only one active OTA update at a time for each client. An active OTA is indicated by the result `RESULT_DEFAULT`. The result for each OTA update is stored in the database (FirmwareUpdate table).

Table 2: Result Codes for an OTA Update

Result	Description
RESULT_DEFAULT	default state
RESULT_SUCCESS	update was successful
RESULT_NO_STORAGE	no storage available
RESULT_OUT_OF_MEM	out of memory
RESULT_CONNECTION_LOST	connection was lost
RESULT_INTEGRITY_FAILED	integrity check failed
RESULT_UNSUP_FW	unsupported firmware
RESULT_INVALID_URI	invalid uri provided
RESULT_UPDATE_FAILED	update failed
RESULT_UNSUP_PROTO	unsupported protocol

Testing with Mock Simulation

The OTA process can be tested without real hardware or a Leshan server using the integrated mock environment.

1. Start the mock environment with `make run-mock`.
2. Upload a firmware file in the dashboard.
3. Select a mock device (e.g., `urn:imei:1`) and initiate an update.
4. The mock simulator will: * Receive the update command via its built-in Mock Leshan API. * Verify the download link by actually fetching the file from the server. * Transition through states with an 11-second delay per step. * Report the new firmware version back to Django.

This allows for rapid verification of the state machine, Celery tasks, and UI updates in a controlled environment.

CHAPTER

THREE

ACCESS CONTROL

flownexus implements a multi-site access control system for organizing devices and controlling user access. This is **not** a multi-tenant SaaS architecture where multiple customers share a single database. Instead, flownexus is designed to run as a single-tenant application on a dedicated server per customer, with Sites providing internal organization for that customer's deployment.

3.1 Deployment Model

flownexus follows a **single-tenant per server** deployment model:

- Each customer runs their own flownexus instance on dedicated infrastructure
- Database isolation is physical — each deployment has its own isolated database
- No shared database or application layer between different customers
- Sites are internal organizational units, not customer boundaries

This architecture provides strong data isolation at the infrastructure level while allowing flexible organization within each customer's deployment.

3.2 Overview

Within a single deployment, the access control system is built around three core concepts:

- **Sites**: Organizational units that group devices and control data visibility
- **Roles**: Hierarchical permission levels (Site User, Site Admin, Global Admin)
- **Permissions**: Granular access to features and operations within a Site

This design allows organizations to organize large device deployments across multiple locations while controlling which users can access which devices.

3.3 Roles and Permissions

The following table shows the access levels for each role:

Feature	Site User	Site Admin	Global Admin
Devices tab	Yes	Yes	Yes
Firmware tab	No	Yes	Yes
Data tab	Yes	Yes	Yes
Permissions tab	No	No	Yes
Manage Firmware	No	Yes	Yes
Perform Operations	No	Yes	Yes
Manage Devices	No	No	Yes
Assign Devices to Sites	No	No	Yes
Create Users	No	No	Yes
Assign Permissions	No	No	Yes

Users are assigned to Sites through **SiteMembership** with specific roles:

Global Admin
: Superuser with full system access including creating users, assigning permissions, and accessing Django admin. Can manage all Sites.

Site Admin
: Full access within their assigned Sites including firmware management and device operations. Cannot create users or reassign devices between Sites from the Permissions tab.

Site User
: Read-only access to dashboards and telemetry within their assigned Sites. Cannot view Firmware tab or perform operations. Typical use: Monitoring personnel, data analysts.

Data visibility is controlled per Site. Users cannot see devices from Sites they don't belong to.

3.4 Site Organization

Within a single flownexus deployment, each **Site** represents an internal organizational grouping — similar to how you might organize resources by building, department, or function within one company:

- **Devices** (Endpoints) belong to exactly one Site
- **Users** can be members of multiple Sites
- **Data** visibility is controlled per Site (users only see their assigned Sites)
- **Firmware** binaries can be shared across Sites within the same deployment
- **Firmware updates** are tracked per Site

Sites help manage large device deployments by grouping related endpoints together. For example:

- **Physical locations**: Building A, Building B, Warehouse North
- **Functional areas**: Production Line 1, HVAC Systems, Security Devices
- **Organizational units**: Engineering Department, Operations Floor

3.5 Feature Permissions

Per-site permissions control access to functional areas:

View Permissions
: Control access to different sections of the web interface:

  - **Devices tab**: View device status and basic information
  - **Firmware tab**: Access firmware management and FOTA updates
  - **Data tab**: Use advanced telemetry filtering and visualization tools

- **Permissions tab**: Global Admin only - manage users and device routing

Operational Permissions
Control what actions users can perform:

- **Manage Firmware**: Upload and delete firmware binaries
- **Perform Operations**: Write resources and execute commands on devices
- **Manage Devices**: Global Admin only for assigning and transferring devices between Sites

3.6 Devices Tab (Read-Only View)

The Devices tab provides a read-only informational view of all devices within the user's accessible Sites. All roles (Site User, Site Admin, Global Admin) can access this tab to view:

- Device status and connectivity state
- Last seen timestamps and telemetry summary
- Basic device information (name, URN, model)
- Current Site assignment

Filtering: Devices are automatically filtered by the user's active Site context. Users only see devices from Sites they are members of. Global Admins can optionally view all devices across all Sites.

Actions: This tab is strictly read-only. Device operations (Write, Execute) and configuration changes are performed from the Firmware tab or device detail views (requires appropriate permissions).

3.7 User Management and Permissions

The Permissions tab serves as the centralized hub for Global Admin routing, providing a unified interface for managing both user assignments and device routing within Sites.

Two-Section Layout:

1. **User Roles Section**: Manage user access and permissions - Create new users and assign initial passwords - Assign users to Sites with specific roles (Site User or Site Admin) - Configure per-site permissions (view tabs, manage firmware, etc.) - Revoke Site memberships
2. **Device Routing Section**: Manage device-to-Site assignments - View unassigned devices (devices with `site__isnull=True`) - Assign devices to specific Sites - Transfer devices between Sites - Bulk assignment operations for efficient device onboarding

This unified approach ensures Global Admins can efficiently manage access control from a single interface without navigating between separate views.

The current implementation keeps device routing and unassigned-device handling in the Global Admin Permissions tab only. Site Admins can work within their assigned Sites, but cannot move devices between Sites through the web UI.

3.7.1 Unassigned Device Handling

Operational State: Unassigned devices (those with `site__isnull=True`) remain fully operational and are not quarantined. They continue to:

- Ingest telemetry data (Resources)
- Trigger events and alerts
- Respond to LwM2M operations
- Maintain normal device communication

Data Visibility: Because unassigned devices have no Site context, their telemetry and events are only visible to Global Admins. Standard users (Site Users and Site Admins) cannot access unassigned device data since they lack the required site membership context. This ensures data isolation while allowing devices to operate during onboarding or site reassignment workflows.

3.7.2 Site Admin User Visibility

Site Admins have limited visibility of other users:

- Can only view users who are members of their Sites
- Cannot see users from other Sites
- Cannot modify user accounts or permissions

3.7.3 User Self-Service

All users can perform limited self-service actions:

- Change their own password via the profile page
- View their current Site memberships
- Switch between assigned Sites using the site selector

3.7.4 Django Admin Fallback

The Django admin interface (`/admin/`) is available as a fallback for advanced administrative tasks. It is restricted to Global Admins only. Regular users and Site Admins cannot access the admin panel. All common user management tasks should be performed through the Permissions tab.

3.8 Site Context and Data Visibility

3.8.1 Site Context Switching

Users belonging to multiple Sites see a site switcher in the top navigation bar. The current Site context filters all data and operations:

- Dashboard shows only devices from the current Site
- Firmware updates are filtered by Site
- Operations only affect devices in the current Site

Navigation menus adapt dynamically based on the user's permissions for the active Site. If a user lacks permission for a feature, the corresponding navigation item is hidden.

3.8.2 Data Visibility

Within a single deployment, the system controls data visibility per Site:

- Users cannot see devices from Sites they don't belong to
- API responses are filtered by Site membership
- Database queries automatically apply Site filters
- Multi-site users see aggregated data when "All Sites" is selected

3.9 Implementation

The access control system is implemented through several components:

3.9.1 Models

Site
: Represents an organizational grouping within a single deployment. Contains name, description, and active status. Endpoints have a ForeignKey to Site. Sites are not used for customer isolation — that is handled by separate server instances.

SiteMembership
: Links users to Sites with roles and permissions. Contains:

- User and Site references
- Role (ADMIN or USER)
- View permissions (overview, firmware, data_analysis)
- Operational permissions (manage_firmware, perform_operations, manage_devices)

3.9.2 Middleware

SiteContextMiddleware
: Sets the current Site context for each request:

- Determines active Site from session or defaults to first available
- Loads user's Site memberships
- Validates user has access to requested Site
- Makes Site context available to views and templates

3.9.3 Context Processors

site_context
: Provides permission variables to all templates:

- `can_view_devices`, `can_view_firmware`, `can_view_data_analysis`
- `can_manage_firmware`, `can_perform_operations`, `can_manage_devices`
- `site_role`, `current_site`, `available_sites`
- `is_global_admin`, `show_all_devices`

3.9.4 Views

Each view checks permissions before processing:

```
def _check_site_permission(request, permission_name):
    if request.user.is_superuser:
        return True
    # Check site membership permissions
    if hasattr(request, 'site_membership') and request.site_membership:
        return getattr(request.site_membership, permission_name, False)
    return False
```

3.9.5 Implementation Hints

The following patterns guide the implementation of the access control views:

Devices List View (Read-Only)

The Devices tab should be implemented as a read-only list view with site-based filtering:

- QuerySet filtering: `Endpoint.objects.filter(site__in=user.site_memberships.values('site'))`

- For Global Admins: optionally show all devices with `Endpoint.objects.all()`
- Remove all POST handling and form logic — this view is strictly for display
- Display device status, connectivity info, and current Site assignment

Permissions View (Global Admin Only)

The unified Permissions tab requires Global Admin access and provides two functional areas:

- **Access Control**: Restrict the entire view using `@user_passes_test(lambda u: u.is_superuser)`
- **Dual-Panel Layout**: Present two distinct sections or sub-tabs:
  - User Roles: Queryset `User.objects.all()` for user management
  - Device Routing: Queryset `Endpoint.objects.all()` with highlighting for unassigned devices (`site__isnull=True`)
- **POST Security**: In assignment handlers, explicitly verify `request.user.is_superuser` and return `HttpResponseForbidden()` if check fails to protect against request spoofing outside the UI

Querying Downstream Models

Models like Resource, Event, and FirmwareUpdate do not have a direct Site foreign key. Always traverse the `endpoint__site` relationship **and** use `select_related` to prevent both data leaks and N+1 query performance bottlenecks:

```
# Correct: Filter by site AND use select_related for performance
resources = Resource.objects.select_related('endpoint__site').filter(
    endpoint__site__in=user.site_memberships.values('site')
)

events = Event.objects.select_related('endpoint__site').filter(
    endpoint__site__in=user.site_memberships.values('site')
)

firmware_updates = FirmwareUpdate.objects.select_related('endpoint__site').filter(
    endpoint__site__in=user.site_memberships.values('site')
)
```

Critical: Omitting the `endpoint__site__in` filter exposes all telemetry to all users (data leak). Omitting `select_related` causes N+1 queries (10,001 queries for 10,000 resources instead of 1).

3.10 Testing with Mock Scenarios

The mock simulation system provides scenario-based configurations for testing access control. See *Devtools & Testing* for details on the multi-site scenario which creates multiple Sites with different user roles and permissions for comprehensive RBAC testing.

CHAPTER

FOUR

APPLICATION GUIDE

This section explains according to best practises how to develop IoT applications with flownexus and Zephyr.

4.1 Endpoint Interactions

flownexus can interact with any endpoint via the LwM2M methods `Read`, `Write`, `Execute` and `Observe`.

4.1.1 Read

Note

Currently not supported

4.1.2 Write

See Chapter *Data Flow: Backend -> Endpoint* for more details.

4.1.3 Execute

Note

Currently not supported

4.1.4 Observe

See chapter *Events via Composite Observe* to see events are generated from composite Observe.

Note

Currently Observe interactions are handled in Leshan, check *LwM2M Observe*.

4.2 Events

IoT devices often trigger events. An event is a significant change in the state of a device or its environment. As an example, there could be a machine that is monitored by an IoT device using multiple sensors. An event is triggered by an unusual combination of sensor readings. It is important to aggregate several states into a single event, so an application in the cloud can analyze the event in detail to know the environmental conditions at that moment.

The database model defines the *Events* table that represents events generated by *endpoints*.

4.2.1 Events via Composite Observe

In order to group multiple resources together in an unambiguous way, LwM2M composite resources are used. Once a composite resource is updated in the IoT device (endpoint), Leshan generates a composite event with all values of the composite resource. Django deserializes the event, stores its individual resources in the database and creates a new event. The individual resources within the event may haveindividual timestamps, the event itself has a separate timestamp in addition.

Note

Currently Observe interactions are handled in Leshan at *LwM2M Observe*.

4.3 Queue Mode

When configuring a energy saving endpoint that uses LTE-M or NB-IoT, queue mode must be enabled in Zephyr so the device will sleep between data transmissions. The Zephyr documentation explains the LwM2M Client in Zephyr in detail.

Listing 1: Enable Queue Mode in Zephyr with a registration update interval of 10 minutes

```
CONFIG_LWM2M_QUEUE_MODE_ENABLED=y
CONFIG_LWM2M_QUEUE_MODE_UPTIME=20
CONFIG_LWM2M_RD_CLIENT_STOP_POLLING_AT_IDLE=y

# Default lifetime is 10 minutes
CONFIG_LWM2M_ENGINE_DEFAULT_LIFETIME=600
```

See chapter *Data Flow: Backend -> Endpoint* to know know more about how queue mode is implemented in flownexus.

4.3.1 eDRX Settings

eDRX (Extended Discontinuous Reception) is a feature that allows the device to sleep for longer periods of time. The device will wake up periodically to check downlink paging messages. If new messages are available, the device will stay awake to receive them. Otherwise, the device will go back to sleep. - Nordic Devzone Article

Setting the eDRX interval to a value significantly smaller than the timeout value of the LwM2M server would theoretically allow flownexus to reach endpoints at any time.

Warning

This approach has not yet been tested! The typical way is to configure endpoints to connect to the server within specific intervals. Inbetween those intervalls, the endpoint will sleep and can't be reached from the server.

4.4 Firmware Setup

4.4.1 LwM2M Version

The used Leshan version uses LwM2M version 1.1. Make sure to enable this version in the Zephyr LwM2M client accordingly.

Listing 2: Enable LwM2M version 1.1 in Zephyr

```
CONFIG_LWM2M_VERSION_1_1=y
```

4.4.2 LwM2M Transport Format

Listing 3: Enable CBOR Format

```
CONFIG_ZCBOR=y
CONFIG_ZCBOR_CANONICAL=y
```

4.4.3 LwM2M SenML Format

Note

Currently not supported in flownexus.

Listing 4: Enable LwM2M version 1.1 in Zephyr

```
# SenML CBOR - default
CONFIG_LWM2M_RW_SENML_CBOR_SUPPORT=y
```

CHAPTER

FIVE

BUILD AND DEPLOY

5.1 Build Setup

The build instructions in the documentation are tested for a native Linux Machine. For MacOS or Windows consider creating a docker container build. One of the developers uses the following *devcontainer.json* build environment:

```
{
  "name": "Ubuntu",
  "image": "mcr.microsoft.com/devcontainers/base:jammy",
  "features": {
              "ghcr.io/devcontainers/features/docker-in-docker:2": {},
              "ghcr.io/devcontainers/features/docker-outside-of-docker:1": {}
      },
  "runArgs": [
    "--cap-add=NET_ADMIN",
    "--cap-add=MKNOD",
    "--device=/dev/net/tun",
    "--sysctl=net.ipv6.conf.all.disable_ipv6=0",
    "--sysctl=net.ipv6.conf.default.disable_ipv6=0"
  ],
  "postCreateCommand": "apt-get update && apt-get install -y iproute2 && echo 'IPv6␣
↪is enabled.'",
  "remoteUser": "root"
}
```

Before you we start with any development here are a few things you should get configured:

- Get the Zephyr SDK downloaded and configured in your root directory. You can find the instructions here.
- Setup a virtual environment for the project.

```
host:~$ sudo apt update && sudo apt upgrade
host:~$ sudo apt install python3-pip python3.10-venv
host:~$ python3.10 -m venv venv
host:~$ source venv/bin/activate
host:~$ pip install --upgrade pip && pip install west
host:~$ mkdir workspace && cd workspace
host:~/workspace$ west init -m https://github.com/flownexus-lwm2m/flownexus --mr main
host:~/workspace$ west update
```

5.2 Container Environment

Both components run in a container. The Leshan server is running in a `openjdk:17-slim` container and the Django server is running in a `python:3.11-slim` container. This allows for an easy and reproducible setup of the server.

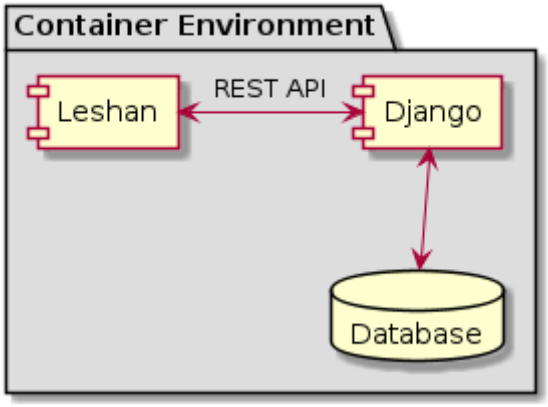


Fig. 1: Both components running in one machine using Podman Compose

The following diagram shows the Container Environment. The file `compose.yml` defines the services and their configuration. The file `Dockerfile.leshan` defines the Leshan container and the file `Dockerfile.django` defines the Django container.

Warning

Make sure to change the password to the admin console as well as other settings like SECRET_KEY, DEBUG flag in a production environment!

The container can be built and started with the following commands:

```
host:~/workspace/flownexus$ make server-build
host:~/workspace/flownexus$ podman-compose -f server/compose.yml up
```

5.3 Setup a Virtual Server

flownexus can be deployed to a virtual server. This chapter explains a basic setup of a virtual server with a domain name. A requirement is to have a Linux server and a domain name. The domain name must point to the server, e.g. via a A/AAAA-Record.

The setup has been tested with a Debian 12 server with a 1C/1GB RAM configuration.

5.3.1 CA and self-signed Certificate

Leshan and the HTTPs download server for firware binaries use self-signed certificates. The flownexus frontend uses certificates that have been issued via Let's Encrypt. The following commands create a self-signed certificate for the domain `flownexus.org`:

Create a Certificate Authority (CA)

1. Generate the CA Private Key:

```
openssl ecparam -genkey -name prime256v1 -out ca.key
```

2. Create a Self-Signed CA Certificate with 100 years validity:

```
openssl req -new -x509 -key ca.key -out ca.crt -days 36500 -subj "/CN=flownexus.
↪org"
```

Create a Server Certificate Signed by the CA

1. Generate the Server Private Key

```
openssl ecparam -genkey -name prime256v1 -out fw_flownexus_org.key
```

2. Generate a Certificate Signing Request (CSR)

```
openssl req -new -key fw_flownexus_org.key -out fw_flownexus_org.csr -subj "/
↪CN=fw.flownexus.org"
```

3. Generate the Server Certificate Signed by the CA

```
openssl x509 -req -in fw_flownexus_org.csr -CA ca.crt -CAkey ca.key -
↪CAcreateserial -out fw_flownexus_org.crt -days 3650 -sha256
```

Generated Files

- `ca.key`: CA private key
- `ca.crt`: CA certificate
- `fw_flownexus_org.key`: Server private key
- `fw_flownexus_org.csr`: Server certificate signing request
- `fw_flownexus_org.crt`: Server certificate signed by the CA

Copy the server certificate and key to the server and store then in `/etc/nginx/ssl/`. Keep the CA certificate and key in a secure location.

5.3.2 Nginx as Reverse Proxy

The following steps show how to configure Nginx as a reverse proxy for the flownexus server. The Nginx server listens on port 443 and forwards the requests to the Django server running on port 8000:

Listing 1: Nginx setup, create Let's Encrypt certificate

```
# Update Sytem, install required packages and enable the firewall
vserver:~/ apt update
vserver:~/ apt install git podman podman-compose nginx certbot python3-certbot-nginx

# Generate a certificate with letsencrypt:
vserver:~/ certbot --nginx -d flownexus.org -d www.flownexus.org
vserver:~/ Create nginx config at /etc/nginx/sites-available/flownexus (see example␣
↪below)
```

Listing 2: Nginx config for the Frontend `/etc/nginx/sites-available/flownexus.org`

```
1 server {
2         listen 443 ssl http2;
3         listen [::]:443 ssl http2;
4         server_name flownexus.org www.flownexus.org;
```

(continues on next page)

(continued from previous page)

```
5
6           error_log /var/log/nginx/flownexus.org.error.log;
7           access_log /var/log/nginx/flownexus.org.access.log;
8           ssl_certificate /etc/letsencrypt/live/flownexus.org/fullchain.pem; # managed␣
    ↪by Certbot
9           ssl_certificate_key /etc/letsencrypt/live/flownexus.org/privkey.pem; #␣
    ↪managed by Certbot
10
11          location / {
12                  proxy_pass http://127.0.0.1:8000/;
13                  proxy_set_header Host $http_host;
14                  proxy_set_header X-Real-IP $remote_addr;
15                  proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
16                  proxy_set_header X-Forwarded-Proto $scheme;
17                  proxy_set_header X-Frame-Options SAMEORIGIN;
18          }
19  }
20
21  server {
22          listen 80;
23          listen [::]:80;
24          server_name flownexus.org www.flownexus.org;
25
26          # Redirect all HTTP requests to HTTPs
27          return 301 https://$host$request_uri;
28  }
```

Listing 3: Nginx config for the https dl Server `/etc/nginx/sites-available/fw.flownexus.org`

```
1   server {
2           listen 443 ssl;
3           listen [::]:443 ssl http2;
4           server_name fw.flownexus.org;
5
6           ssl_certificate /etc/nginx/ssl/fw_flownexus_org.crt;
7           ssl_certificate_key /etc/nginx/ssl/fw_flownexus_org.key;
8
9           location /binaries {
10                  root /var/www/flownexus/;
11                  # Debug option: uncomment to list files
12                  # autoindex on;
13          }
14  }
15
16  server {
17       listen 80;
18       server_name fw.flownexus.org;
19
20       # Redirect all HTTP requests to HTTPS
21       return 301 https://$host$request_uri;
22  }
```

After creating the Nginx config, activate the config and restart the Nginx.

Listing 4: Activate the Nginx config

```
# Activate the Nginx config:
vserver:~/ ln -s /etc/nginx/sites-available/flownexus.org /etc/nginx/sites-enabled/
vserver:~/ ln -s /etc/nginx/sites-available/fw.flownexus.org /etc/nginx/sites-enabled/

# Test the Nginx config:
vserver:~/ nginx -t

# Restart Nginx:
vserver:~/ systemctl restart nginx
```

5.3.3 Test the Download Server

If you have setup an A/AAAA-Record, you can now test the download server. It is available at https://fw.flownexus.org/binaries. If you uncomment the option `autoindex on;` in the Nginx config, you can list the files in the directory.

5.3.4 Start flownexus

After the setup, download flownexus and start it with using podman-compose in detached mode. Make sure to change the `DEPLOY_SECRET_KEY` and `DEBUG` flag in the `settings.py` file before deploying.:

Listing 5: Start flownexus with podman-compose

```
vserver:~/ git clone https://github.com/flownexus-lwm2m/flownexus.git
# Change the DEPLOY_SECRET_KEY and DEBUG flag in the settings.py file
vserver:~/flownexus$ make server-build
vserver:~/flownexus$ podman-compose -f server/compose.yml up -d
```

flownexus is now available at https://flownexus.org. The server is running in a Docker container and the Nginx server is used as a reverse proxy.

5.3.5 Security Considerations

Consider enabling the firewall and only keep required ports open:

- **Port 80, TCP**: HTTP
- **Port 443, TCP**: HTTPS
- **Port 22, TCP**: SSH
- **Port 5683, UDP**: CoAP

Warning

flownexus is not production ready. This server setup is only intended for testing purposes.

The current flownexus configuration uses the default Django `DEPLOY_SECRET_KEY` and enables the `DEBUG` flag. This is a security risk and must be change before deploying.

Currently, the default django inbuild webserver is used. This is not recommended for production use. Consider using a production-ready webserver like Nginx or Apache.

CHAPTER

SIX

DEVTOOLS & TESTING

flownexus provides development and testing tools in the `devtools/` directory. The documentation is organized by test level, from fast unit tests to comprehensive end-to-end testing.

6.1 Testing

The project provides several Makefile targets for testing. These commands can be run from the repository root and ensure consistent test environments.

6.1.1 Compliance Checks

Run static analysis and formatting checks:

```
host:~/flownexus$ make compliance
```

This performs:

- Code formatting validation (black, isort)
- Linting (flake8, ruff)
- Git history validation (signed commits)

All checks must pass before submitting changes.

6.1.2 Unit Tests

Run Django unit tests in an isolated Tox environment:

```
host:~/flownexus$ make test-django
```

This executes all unit tests using tox, which ensures tests run in a clean, reproducible environment with proper dependencies. Unit tests cover:

- REST API endpoint validation
- Data deserializer functionality
- Model relationships and constraints
- Permission and RBAC logic

You can also run tests directly (without tox) for faster iteration during development:

```
host:~/flownexus/server/django$ python manage.py test sensordata
```

6.1.3 Integration Testing

The `make test-django` command also runs integration tests, including the mock simulation integration test:

```
host:~/flownexus/server/django$ python manage.py test sensordata.tests.integration.
↪test_mock_simulation
```

This test uses Django's `LiveServerTestCase` to spin up a real HTTP server and verify that the simulation backend can successfully register devices and ingest data.

6.1.4 End-to-End Tests

Run the full E2E test suite including Zephyr simulation:

```
host:~/flownexus$ make test-e2e
```

This builds Zephyr binaries, starts the server stack, runs the simulation, and verifies data ingestion. Requires Podman and Zephyr toolchain.

6.1.5 Run All Tests

Execute the complete test suite:

```
host:~/flownexus$ make test-all
```

This runs compliance checks, unit tests, and E2E tests in sequence.

6.2 Development Tools

For iterative development and debugging, flownexus provides simulation tools that emulate IoT devices at different fidelity levels.

6.2.1 Mock Simulation

The Mock backend is a lightweight Python-based simulator that interacts directly with the Django Ingestion API. It generates synthetic device data (temperature, humidity) with sine-wave patterns for easy visualization testing and simulates the complete FOTA lifecycle.

Key features:

- **Zero Dependencies**: No Docker, Podman, or Zephyr toolchains required
- **Mock Leshan API**: Built-in REST API on port 8081 mimics the Leshan server
- **Sine-wave Data**: Smooth curves make frontend chart verification easy
- **Integrated FOTA**: Simulates firmware update state transitions

The recommended way to use the Mock backend is via the integrated development environment:

```
host:~/flownexus$ make run-mock
```

This command orchestrates Redis, Django, Celery, and the Mock Simulation in a single step. The environment uses a temporary database in `/tmp/` that is deleted on shutdown. Use `-v` for verbose output.

Alternatively, run the simulator standalone against an existing server:

```
host:~/flownexus$ python3 devtools/mock/run.py --config devtools/mock/config.yaml
```

Scenario-Based Configuration

The mock environment supports YAML-based scenarios for testing different features. Scenarios are stored in `devtools/mock/scenarios/`:

- **default**: 5 devices, 1 site, single admin user (`make run-mock`)
- **multi-site**: 15 devices, multiple sites/users with RBAC (`make run-mock-multi-site`)

Create custom scenarios by adding YAML files. Each scenario defines devices, sites, and users for reproducible test environments.

Example Mock Configuration (`devtools/mock/config.yaml`):

```yaml
type: mock
url: "http://localhost:8000/flownexus/ingest"
device_count: 5
interval: 2.0  # Seconds between updates
duration: 0    # 0 = Run forever
run: true      # Automatically start simulation
```

6.2.2 Zephyr Simulation

The Zephyr backend runs actual Zephyr OS firmware binaries in a containerized network environment. This provides high-fidelity simulation for end-to-end testing and LwM2M protocol compliance verification.

Prerequisites: Podman, Python `docker`/`PyYAML` libraries, and Zephyr toolchain.

```
host:~$ apt install podman podman-compose
host:~$ pip install docker PyYAML
```

The script expects the `server_mynetwork` Docker network to be available (from the running server stack).

CLI Usage:

Listing 1: Usage of devtools/zephyr/run.py

```
host:~/flownexus$ python3 devtools/zephyr/run.py --help
usage: run.py [-h] --config CONFIG [--build] [--run] [--local]

Flownexus Zephyr Device Simulator

options:
  -h, --help       show this help message and exit
  --config CONFIG  Path to simulation YAML config file (Required)
  --build          Build Zephyr binaries
  --run            Run simulation
  --local          Use local Leshan server
```

Example Zephyr Configuration (`devtools/zephyr/config.yaml`):

```yaml
type: zephyr
device_count: 1
local_leshan: true  # Connect to the Leshan container
verbose: false
delay: 0            # Startup delay between instances (ms)
build: false
run: false
```

Running the Simulation:

```
# Build binaries (required once or after code changes)
host:~/flownexus$ make build-sim

# Run simulation against local server
```

(continues on next page)

(continued from previous page)

```
host:~/flownexus$ python3 devtools/zephyr/run.py --config devtools/zephyr/config.yaml␣
↪--run --local
```

6.3 External Resources

See also

- Zephyr LwM2M API
- Zephyr LwM2M Client Sample
- Zephyr Native Sim Board
- Zephyr Networking with Multiple Instances

CHAPTER

SEVEN

DOCUMENTATION

7.1 Build the documentation

```
host:~$ sudo apt-get install default-jre plantuml graphviz
host:~$ source venv/bin/activate
host:~$ cd workspace/flownexus/doc
host:~/workspace/flownexus/doc$ pip install -r requirements.txt
host:~/workspace/flownexus/doc$ tox -e html
```

Open the generated index.html in the doc/build directory in your browser.

CHAPTER

EIGHT

GLOSSARY

Terms are usually explained and indexed directly in the documentation chapters. Additionally, some general terms are defined here to provide a quick reference. The terms defined here, as well as the indexed terms from the documentation chapters can be referenced in the documentation by using the `:term:` role.

8.1 Additional Terms and Definitions

LwM2M
: Lightweight Machine to Machine protocol.

Leshan
: LwM2M server implementation as one main component of the system.

Backend
: Django instance that is responsible for the REST API, database and visualization.

Frontend
: Web application that visualizes the data from the backend, integrated into Django.

Object
: A logical grouping of one or multiple Resources in LwM2M. An Object is identified by an Object ID.

CHAPTER

NINE

INTERNAL DJANGO

The API Documentation is available at https://flownexus-lwm2m.github.io/flownexus/.

CHAPTER

TEN

WEBLOG

10.1 Release v0.1.0

#3

Date
Aug 4, 2024

Author
https://github.com/jonas-rem

Tags
Release Notes

First release of the project. This is a development release.

- Created a flownexus logo.
- Generic database model for LwM2M objects.
- Added a frontend mockup based on AtminLTE.
- ReST APIs inbetween Leshan and Django.
- Asynchronous task queue for Endpoint interactions from Django via Celery.
- Bidirectional communication between L2M2M endpoints and Django via Leshan.
- Subscribe to single and composite Observations in Leshan statically.
- OTA Update functionality on the server side. Full-functioning firmware example pending.

10.2 Update Documentation & Add Weblog

#2

Date
May 15, 2024

Author
https://github.com/Kappuccino111

Tags
Add, Update

- Switched the new documentation to *Sphinx-Book-Theme* for better readability and navigation.
- Added a new section for the weblog to keep track of updates and changes in the project.

10.3 Project Starts

#1

Date
January 1, 2024

Author
https://github.com/jonas-rem

Tags
Update

- The project has started and the initial documentation has been added.

INDEX